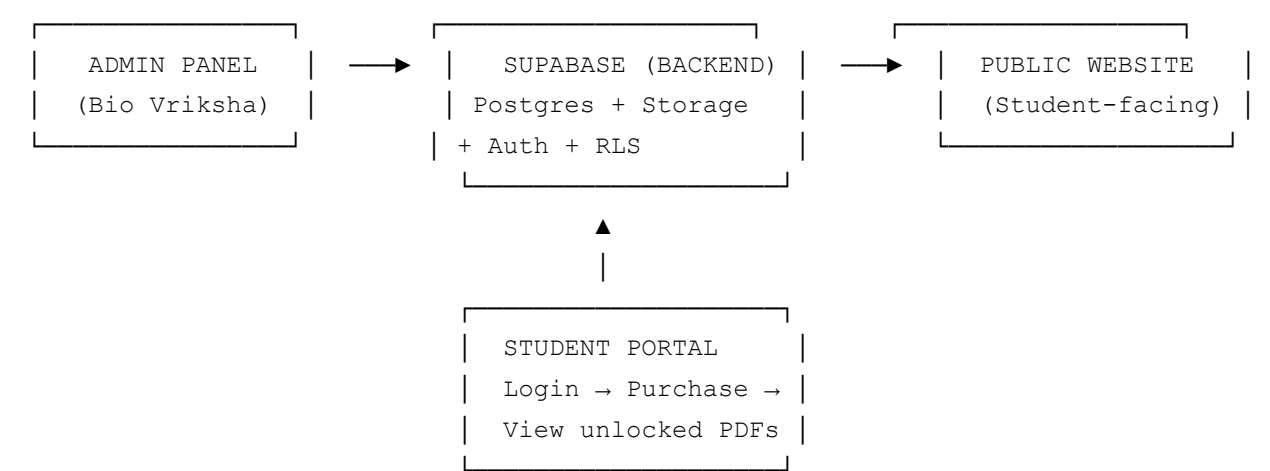


Bio Vriksha — Backend, Admin Panel & CMS Guide (Supabase)

Prepared by Bug Byte · For BioVriksha EdTech Platform Stack: Supabase (Postgres + Auth + Storage + RLS) + your frontend (React/Next.js)

1. How Everything Connects (Big Picture)



Core idea: Admin Panel and Website are two different frontends talking to the **same Supabase project**. The moment Admin uploads/edits a PDF or creates a Chapter, it writes a row into a Postgres table. The public website reads from that same table (`select * from pdfs where is_active = true`) — so it appears live, with **no manual redeploy or sync step needed**. This is the “connected logic” you asked for.

2. Database Schema (Postgres tables in Supabase)

2.1 `profiles` (extends Supabase Auth users)

Every login (admin or student) creates a row here automatically via a trigger.

Column	Type	Notes
id	uuid (PK, = auth.users.id)	linked to Supabase Auth

full_name	text	student/admin name
phone	text	student's phone number
role	text	'admin' or 'student'
created_at	timestampz	default now()

```
create table profiles (
  id uuid primary key references auth.users(id) on delete cascade,
  full_name text,
  phone text,
  role text not null default 'student' check (role in ('admin','student')),
  created_at timestampz default now()
);
```

2.2 chapters

Admin creates these first — this is what populates the “Select Chapter” dropdown.

```
create table chapters (
  id uuid primary key default gen_random_uuid(),
  name text not null,           -- e.g. "Human Reproduction"
  subject text default 'Biology', -- kept flexible for future subjects
  order_index int default 0,    -- controls display order on site
  is_active boolean default true,
  created_at timestampz default now()
);
```

2.3 pdfs

Every uploaded PDF (free or paid) is one row here.

```
create table pdfs (
  id uuid primary key default gen_random_uuid(),
  chapter_id uuid references chapters(id) on delete set null,
  title text not null,
  description text,
  thumbnail_url text,          -- public image URL (bucket: pdf-thumbnails)
  file_path text not null,     -- private file path (bucket: pdf-files)
  is_free boolean default false,
  price numeric(10,2) default 0, -- ignored if is_free = true
  is_active boolean default true, -- admin can hide/disable without deleting
  page_count int,
```

```
    created_at timestamptz default now(),
    updated_at timestamptz default now()
);
```

2.4 purchases (orders)

One row per successful student purchase — this is your “Orders” list.

```
create table purchases (
  id uuid primary key default gen_random_uuid(),
  student_id uuid references profiles(id) on delete cascade,
  pdf_id uuid references pdfs(id) on delete cascade,
  amount_paid numeric(10,2) not null,
  payment_status text default 'success' check (payment_status in ('success','fail')),
  payment_gateway_id text,      -- Razorpay/Cashfree transaction id
  purchased_at timestamptz default now(),
  unique (student_id, pdf_id) -- can't buy the same PDF twice
);
```

That’s the entire core schema — 4 tables. Everything else (admin dashboard stats, revenue, “who bought what”) is just **queries/views** on top of these.

3. Storage Buckets

Supabase Storage holds the actual files. Create two buckets:

Bucket	Access	Holds
pdf-thumbnails	Public	Cover images for cards on the website
pdf-files	Private	Actual PDF files — never publicly listable

Because `pdf-files` is private, nobody can view/download a paid PDF just by guessing the URL. Access is only granted through a **signed URL** generated after checking the `purchases` table (explained in §5).

4. Row Level Security (RLS) — the real “logic” layer

RLS policies live inside Supabase and enforce rules at the database level, not just in your app code — so even if someone bypasses your frontend, the database itself refuses

unauthorized access.

```
-- Enable RLS
alter table chapters enable row level security;
alter table pdfs enable row level security;
alter table purchases enable row level security;
alter table profiles enable row level security;

-- Anyone (even logged-out visitors) can read active chapters & pdf metadata
create policy "public read chapters"
  on chapters for select using (is_active = true);

create policy "public read pdfs"
  on pdfs for select using (is_active = true);

-- Only admins can insert/update/delete chapters & pdfs
create policy "admin manage chapters"
  on chapters for all using (
    exists (select 1 from profiles where id = auth.uid() and role = 'admin')
  );

create policy "admin manage pdfs"
  on pdfs for all using (
    exists (select 1 from profiles where id = auth.uid() and role = 'admin')
  );

-- Students can see only their own purchases; admin can see all
create policy "student sees own purchases"
  on purchases for select using (
    student_id = auth.uid()
    or exists (select 1 from profiles where id = auth.uid() and role = 'admin')
  );

-- Only the system (via a secure edge function / backend) inserts purchases after
create policy "no direct client insert on purchases"
  on purchases for insert with check (false);
```

Note the last policy: purchases are **never inserted directly from the browser**. They're inserted by a server-side Supabase Edge Function after your payment gateway (Razorpay/Cashfree) confirms payment — this prevents someone from faking a "successful purchase" from dev tools.

5. Admin Panel — Exact Flow You Asked For

Step 1 — Create Chapter Admin Panel → “Chapters” tab → + New Chapter → enters name (e.g. “Genetics & Evolution”) → Save → `insert into chapters (name) values (...)`

Step 2 — Upload PDF Admin Panel → “Upload PDF” form:

1. **Select Chapter** — dropdown populated live from `chapters` table
2. **Title** — text input
3. **Thumbnail** — image upload → goes to `pdf-thumbnails` bucket → returns public URL
4. **Description** — textarea
5. **Upload PDF file** — goes to `pdf-files` bucket (private) → returns file path
6. **Free / Paid toggle**
 - If Free → `is_free = true` , price field hidden
 - If Paid → `is_free = false` , price input required (e.g. ₹10)
7. Save → `insert into pdfs (...)`

Step 3 — Manage Existing PDFs A table/grid view of all PDFs with:

- Edit title, description, price, free/paid status
- Toggle **Active/Inactive** (instantly hides/shows it on the live site — no delete needed)
- Delete permanently
- Filter by chapter

Because the website reads `pdfs where is_active = true` directly from Supabase, **any change here reflects on the live website within seconds** — that’s the “connected” behaviour you wanted.

6. Student Portal — View-Only Access (No Download)

This is the part that enforces “unlock to view on website only, never downloadable”:

1. Student clicks **Unlock ₹10** on a paid PDF.
2. Payment gateway (Razorpay/Cashfree) checkout opens.
3. On payment success, a **Supabase Edge Function** verifies the payment signature server-side, then inserts a row into `purchases` .

4. Student is redirected to the PDF viewer page.
5. Viewer page calls another **Edge Function**: `get-pdf-url`, which:
 - Checks `purchases` table (or `is_free = true`) to confirm the student is allowed to view this PDF.
 - If allowed, generates a **signed URL** from the private `pdf-files` bucket with a **short expiry** (e.g. 5 minutes).
 - Returns that signed URL only — never the permanent file path.
6. Frontend loads the PDF inside an **embedded viewer** (e.g. `pdf.js` in an `iframe`) — no `<a download>` link is ever rendered, and the toolbar's native download/print button is disabled in the viewer config.
7. If the student refreshes or reopens later, a fresh signed URL is generated (previous one has already expired) — but only if they still own that purchase.

This gives you real “view-only, unlockable” behaviour without ever exposing a permanent downloadable link.

7. Admin CMS Dashboard (Orders, Students, Revenue)

Since `purchases` already links `student_id` → `profiles` and `pdf_id` → `pdfs`, your dashboard is just SQL views:

```
-- Full order list for the admin dashboard
select
  p.full_name as student_name,
  p.phone as student_phone,
  pdf.title as pdf_title,
  pur.amount_paid,
  pur.payment_status,
  pur.purchased_at
from purchases pur
join profiles p on p.id = pur.student_id
join pdfs pdf on pdf.id = pur.pdf_id
order by pur.purchased_at desc;

-- Revenue summary
select
  sum(amount_paid) as total_revenue,
  count(*) as total_orders
```

```
from purchases
where payment_status = 'success';

-- Registered students list
select full_name, phone, created_at
from profiles
where role = 'student'
order by created_at desc;
```

Build these as three simple admin screens: **Orders, Students, Revenue/Analytics** — this makes the Admin Panel function as a full mini-CMS, exactly like you described.

8. Authentication Setup

- Use **Supabase Auth** (email/password, or phone+OTP for students if preferred).
- On signup, a Postgres trigger auto-creates the matching `profiles` row with `role = 'student'` by default.
- Admin accounts are created manually (by you) directly in Supabase, then their `profiles.role` is updated to `'admin'` via SQL — students can never self-promote to admin since RLS only checks the `role` column, which they have no write access to (no update policy for `profiles.role` is granted to students).

```
create policy "students can only update their own non-role fields"
on profiles for update using (id = auth.uid())
with check (id = auth.uid());
-- role column changes should be restricted at the application layer / via a sepa
```

9. Suggested Build Order

1. Set up Supabase project → create the 4 tables + 2 storage buckets (§2, §3)
2. Apply RLS policies (§4)
3. Build Admin Panel: Chapters CRUD → PDF Upload/Manage → Orders/Students dashboard (§5, §7)
4. Build Student Portal: Register/Login → Browse chapters/PDFs → Purchase flow → View-only viewer (§6)

5. Connect public website's Recent/Paid PDF sections to read live from `pdfs` (already matches your homepage draft design)
 6. Wire up payment gateway (Razorpay recommended for low-value ₹10+ transactions) via Edge Function
-

This document is a technical blueprint — happy to turn any section (e.g. the Admin Panel UI screens, or the exact Edge Function code for payment verification) into the next build step whenever you're ready.